# Priority-Guided Masking for Diffusion Language Models

**Austin Feng**
CPSC 477
Yale University
New Haven, CT 06510
austin.feng@yale.edu

**Aniketh Malyala**
CPSC 477
Yale University
New Haven, CT 06510
aniketh.malyala@yale.edu

**Andrew Xu**
CPSC 477
Yale University
New Haven, CT 06510
andrew.xu@yale.edu

## Abstract

Diffusion Language Models (DLMs) offer a promising alternative to traditional autoregressive models by enabling parallel generation and flexible generation orders. This makes it ideal for structured generation, efficient language models, and iterative reasoning. However, current state-of-the-art DLMs such as LLaDA typically employ a uniform masking strategy during pretraining, which may not be optimal as certain tokens (e.g. keywords) carry more semantic weight than others. We propose an IDF-based Priority-Guided Masking (PGM) strategy for DLMs, hypothesizing that prioritizing the generation of important tokens first can enhance model performance. Our results indicate potential promise in generating keywords early, comparing LLaDA-Instruct fine-tuned using PGM to uniform masking. Our code can be found on GitHub for reproducibility. [1]

## 1 Introduction

Diffusion models have demonstrated remarkable success in generative tasks for continuous data domains like images and audio [6, 16]. Recently, efforts have been made to adapt these models for discrete data, particularly natural language [12, 13, 15, 17]. Diffusion Language Models (DLMs) operate by iteratively masking discrete tokens in a forward process and learning to unmask them in a reverse generative process. Unlike autoregressive models that generate tokens sequentially, DLMs can operate on the entire sequence in parallel, offering potential advantages in inference speed, structured generation, and error correction.

State-of-the-art DLMs such as LLaDA [13] and commercial models such as Mercury Coder [5] have shown competitive performance relative to strong autoregressive baselines, sometimes with significantly less training data. LLaDA 8B, for instance, achieves comparable performance to LLaMA3 8B despite using significantly fewer training tokens (2.3T vs 15T). These models typically employ a uniform masking strategy: during pretraining, tokens are selected with equal probability at each step of the forward diffusion process.

### 1.1 Project Motivation

Uniform masking is simple and theoretically tractable. Intuitively, however, some words contribute more significantly to the overall meaning and structure of an output than others. We hypothesize that guiding the model to generate these high-impact tokens earlier in the reverse process could lead to a more coherent and efficient generation process, giving the model a stronger scaffold to fill in the remaining tokens accurately. This method also poses as a potential advantage of DLMs over ARMs

[1] Our code can be found at `https://github.com/ayfeng23/llada_custom_scheduler`. Details in Appendix D.

Preprint. Under review.

as ARMs must generate in a left-to-right autoregressive manner while DLMs can choose an optimal order of generation.

In this work, we introduce Priority-Guided Masking (PGM), a modification to the usual DLM training scheme. Instead of uniform masking, PGM assigns a priority score to each token and biases the masking process such that high-priority tokens are less likely to be masked early (and thus are predicted earlier during generation). We use IDF scores as a proxy for token importance and modify the masking schedule using a Beta distribution parameterized by these scores. We evaluate our approach by fine-tuning the LLaDA-Instruct model using PGM and uniform masking and comparing the results.

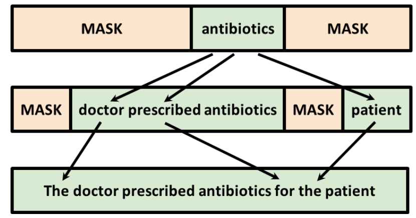


Figure 1: PGM can encourage DLMs to generate predictive words like "antibiotics" that inform other medical terms such as "prescribed" or "patient."

### 1.2 Related Work

**Diffusion Models for Language:** Adapting diffusion models from continuous domains (images) to discrete domains (text) poses challenges. Early approaches involved embedding text into continuous spaces [10] or using discrete corruption processes [2]. More recent models, like LLaDA [13], utilize a mask-and-predict framework directly in the discrete token space, analogous to Masked Language Models (MLMs) like BERT [4] but framed within a diffusion process. LLaDA employs a sequential masking schedule where tokens are masked uniformly at random over diffusion timesteps. Our work builds directly on LLaDA by modifying this uniform masking schedule. Commercial interest is also growing, exemplified by Inception Labs' Mercury Coder [5], a DLM optimized for speed and cost-efficiency in coding tasks.

**Non-Uniform Masking in Language Models:** While uniform masking is standard in many MLMs and DLMs, some research has explored non-uniform strategies. For instance, some MLM pre-training techniques focus masking on more informative tokens or entities [8]. In the context of autoregressive models, techniques like targeted fine-tuning or weighting rarer words have been explored to improve performance on specific tasks or handle long-tail phenomena [9]. Our work applies the concept of non-uniformity to the masking schedule of DLMs, specifically using IDF as a signal for token importance to guide the diffusion training process.

## 2 Methods

Our proposed approach modifies the forward masking process of the LLaDA model to incorporate token priorities.

### 2.1 Baseline: Uniform Masking in LLaDA

The standard LLaDA training involves a forward process where tokens $x_0$ are gradually masked over $T$ timesteps to produce $x_T$. At each step $t$, a subset of the currently unmasked tokens are replaced with a special [MASK] token. In the original formulation [13], the probability of a token being masked by time $t$ follows a uniform schedule. Specifically, each token $i$ is assigned a random time $u_i \sim \text{Unif}(0, 1)$. A mask ratio $p_t$ (e.g., $p_t = t/T$) is defined for timestep $t$, and token $i$ is masked if $u_i < p_t$. This results in a uniform probability of any given token being masked over the full process.

### 2.2 Priority-Guided Masking (PGM)

We propose to make the masking probability dependent on the semantic importance or "priority" of a token. Our goal is to ensure that high-priority tokens remain unmasked for longer durations in the forward process, meaning they are predicted earlier (closer to $t = 0$) in the reverse generative process.

Let $p(x_i) \in [0, 1]$ be the priority score assigned to token $x_i$. We modify the sampling of the masking time $u_i$ for each token. Instead of $u_i \sim \text{Unif}(0, 1)$, we sample $u_i$ from a distribution whose mean is correlated with the priority $p(x_i)$. Specifically, we use the Beta distribution:

$$u_i \sim \text{Beta}(\alpha_i, \beta_i) \tag{1}$$

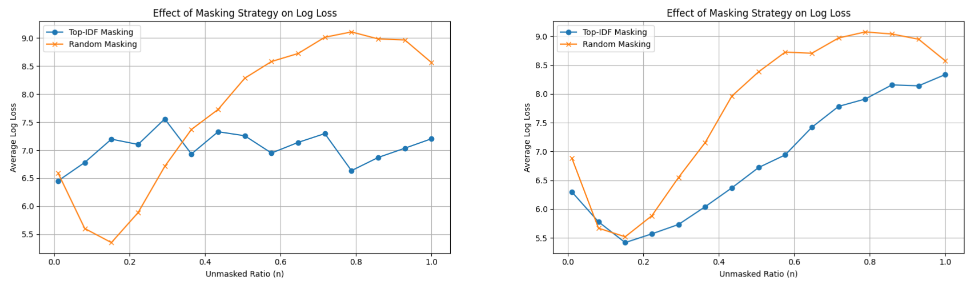


Figure 2: The effect of masking strategy on log loss, based on the proportion of tokens still unmasked. The left figure compares masking top-IDF tokens to random masking, while the right figure compares masking top-IDF tokens with 50% random resample compared to the random baseline.

where the parameters $\alpha_i$ and $\beta_i$ are functions of the priority $p(x_i)$. We want the mean of the distribution, $\mathbb{E}[u_i] = \frac{\alpha_i}{\alpha_i+\beta_i}$, to increase with priority. We achieve this using the following formula/

$$u_i \sim \text{Beta}(1 + \gamma p(x_i), 1 + \gamma(1 - p(x_i))) \tag{2}$$

Here, $\gamma > 0$ is a hyperparameter controlling the concentration of the distribution (higher $\gamma$ makes the distribution more peaked around its mean). The mean of this distribution is $\mathbb{E}[u_i] = \frac{1+\gamma p(x_i)}{2+\gamma}$. This mean scales linearly from $\frac{1}{2+\gamma}$ (for $p(x_i) = 0$) to $\frac{1+\gamma}{2+\gamma}$ (for $p(x_i) = 1$). Crucially, tokens with higher priority $p(x_i)$ will have a higher mean $u_i$, making them less likely to be masked early (i.e., less likely to satisfy $u_i < p_t$ for small $t$). Figure 3 in Appendix A.1 illustrates how the Beta distribution shifts for different priorities. We use Inverse Document Frequency as a proxy to assign a priority score $p(x_i)$ to each token in the vocabulary. Details on assigning priority scores can be found in Appendix A.2.

## 3 Results

We conducted experiments to validate the core hypothesis and evaluate the effectiveness of PGM during fine-tuning.

### 3.1 Exploratory Analysis: Impact of Keyword Generation Order

To initially test the claim that generating keywords first is beneficial, we performed inference-time analysis on the pre-trained LLaDA model without fine-tuning. We created partially masked sequences using different strategies: (1) Random masking (baseline), (2) Masking top-IDF tokens first, (3) Masking top-IDF tokens first but resampling 50% of the masked tokens randomly. Here, (3) is similar to our setting, as we don't strictly mask the top-IDF tokens but the stochasticity of scores from our Beta distribution and temperature parameter simulates resampling. We measured the model's log loss on the remaining masked tokens to evaluate performance.

Our results (see Figure 2) indicate that (3) nearly always has lower log-loss than (1), and (2) has significantly lower log-loss than (1) when more tokens are unmasked.

### 3.2 Fine-tuning Results

We evaluate PGM by fine-tuning the LLaDA-Instruct model on a subset of 60k samples from the Smoltalk instruction dataset [1] with inverse sigmoid scheduled sampling using LoRA (with $r = 64$ and $\alpha = 64$) [7]. We hypothesize that PGM-based DLMs will have more errors when initially unmasking, as we predict keywords first, which are usually harder to generate. So we modify the data distribution to contain more samples that are highly masked. Specifically, LLaDA originally samples the mask ratio $p_t \sim \text{Unif}(0, 1)$ and we modify this by resampling 25% of the time to draw $p_t \sim \text{Unif}(0.8, 1)$. We compare PGM and uniform masking using the same fine-tuning pipeline. Additional details about computing infrastructure, dataset, and hyperparameters and tuning process are detailed in Appendix B. Our training loss curves can be found in Appendix C.1.

| Model | Validation Loss | Validation Perplexity |
|---|---|---|
| Scheduled Masking | **4.854** | **128.313** |
| Baseline | 5.222 | 185.553 |

| Low Confidence | BLEU | ROUGE-1 | ROUGE-L | BERTScore-F1 |
|---|---|---|---|---|
| Pretrained Baseline | 0.0138 | 0.0957 | 0.0833 | – |
| Fine-tuned Baseline | 0.1543 | 0.4989 | 0.3873 | 0.8923 |
| Ours | **0.1673** | **0.5060** | **0.3910** | **0.8931** |

| Random | BLEU | ROUGE-1 | ROUGE-L | BERTScore-F1 |
|---|---|---|---|---|
| Pretrained Baseline | 0.0109 | 0.0962 | 0.0824 | – |
| Fine-tuned Baseline | 0.1237 | 0.4785 | 0.3721 | 0.8898 |
| Ours | **0.1473** | **0.4954** | **0.3887** | **0.8932** |

Table 1: Results comparing the baseline to our method (scheduled masking). The top table shows that our method results in lower loss and perplexity on our validation set after training. The next two tables show that our model, using both low-confidence and random remasking, outperforms the baseline on BLEU, ROUGE-1, ROUGE-L, and BERTScore-F1.

To evaluate the model, we first compare the loss and perplexity of our model and the fine-tuned baseline on a validation subset from Smoltalk. Then, we compare our model using two different remasking strategies, detailed in Appendix C.2, against our baseline on a variety of metrics, including BLEU [14], ROUGE-1, ROUGE-L [11], and BERTScore-F1 [18]. Results are detailed in Table 3.2.

Our results indicate that across all benchmarks we evaluated, our model performs slightly better than the baselines. It seems that using PGM, our model gains a slightly better intrinsic understanding of language. However, there is not a large enough difference between our results and the fine-tuned baseline for us to consider our results significant.

## 4 Conclusion and Future Work

In this work, we present Priority-Guided Masking (PGM), an approach for training Diffusion Language Models that biases the masking process based on token importance, proxied by IDF scores. By modifying the masking schedule using a Beta distribution, we encourage the model to generate high-priority tokens earlier. We fine-tuned LLaDA-Instruct using PGM and compared it to a standard uniform masking baseline.

Our results suggest that modifying the masking strategy in DLMs based on token priority is a viable approach. The preliminary log-loss experiments indicated a potential benefit to revealing high-IDF tokens earlier. The improvement observed support our hypothesis that guiding the token generation order towards semantically important tokens can be beneficial to the model in the subsequent infilling process. We believe that our results are not strong enough to be the new state-of-the-art, but indicate that improving DLMs through this avenue of exploration is possible.

This preliminary work opens several avenues for future research in priority-guided masking:

- **Alternative Priority Metrics:** Explore more sophisticated priority scores beyond IDF, such as those based on part-of-speech tags, syntactic roles, learned importance scores (potentially using attention mechanisms), or context-dependent measures.
- **Hyperparameter Tuning:** Investigate different settings for the Beta distribution concentration ($\gamma$), LoRA parameters, learning rates, and the scheduled sampling strategy.
- **Remasking Strategies:** Experiment with different remasking or infilling strategies during inference that explicitly leverage the priority information learned during training.
- **Domain-Specific Tuning:** Apply PGM to domain-specific fine-tuning (e.g., medical, legal texts) where accurately generating key terms early might be particularly crucial.

Exploring non-uniform strategies like PGM could further unlock the potential of diffusion models for complex language generation tasks.

## References


[1] Loubna Ben Allal et al. *SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model*. 2025. arXiv: 2502.02737 [cs.CL]. URL: https://arxiv.org/abs/2502.02737.

[2] Jacob Austin et al. *Structured Denoising Diffusion Models in Discrete State-Spaces*. 2023. arXiv: 2107.03006 [cs.LG]. URL: https://arxiv.org/abs/2107.03006.

[3] Marco Cornolti. *Wikipedia Document Terms Frequency*. URL: https://github.com/marcocor/wikipedia-idf.

[4] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 [cs.CL]. URL: https://arxiv.org/abs/1810.04805.

[5] Stefano Ermon, Aditya Grover, and Volodymyr Kuleshov. Feb. 2025. URL: https://www.inceptionlabs.ai/introducing-mercury.

[6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239 [cs.LG]. URL: https://arxiv.org/abs/2006.11239.

[7] Edward J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. arXiv: 2106.09685 [cs.CL]. URL: https://arxiv.org/abs/2106.09685.

[8] Mandar Joshi et al. *SpanBERT: Improving Pre-training by Representing and Predicting Spans*. 2020. arXiv: 1907.10529 [cs.CL]. URL: https://arxiv.org/abs/1907.10529.

[9] Frederik Kunstner et al. *Heavy-Tailed Class Imbalance and Why Adam Outperforms Gradient Descent on Language Models*. 2024. arXiv: 2402.19449 [cs.LG]. URL: https://arxiv.org/abs/2402.19449.

[10] Xiang Lisa Li et al. *Diffusion-LM Improves Controllable Text Generation*. 2022. arXiv: 2205.14217 [cs.CL]. URL: https://arxiv.org/abs/2205.14217.

[11] Chin-Yew Lin. "ROUGE: A Package for Automatic Evaluation of Summaries". In: *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, July 2004, pp. 74–81. URL: https://aclanthology.org/W04-1013/.

[12] Aaron Lou, Chenlin Meng, and Stefano Ermon. *Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution*. 2024. arXiv: 2310.16834 [stat.ML]. URL: https://arxiv.org/abs/2310.16834.

[13] Shen Nie et al. *Large Language Diffusion Models*. 2025. arXiv: 2502.09992 [cs.CL]. URL: https://arxiv.org/abs/2502.09992.

[14] Kishore Papineni et al. "BLEU: a method for automatic evaluation of machine translation". In: *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*. ACL '02. Philadelphia, Pennsylvania: Association for Computational Linguistics, 2002, pp. 311–318. DOI: 10.3115/1073083.1073135. URL: https://doi.org/10.3115/1073083.1073135.

[15] Subham Sekhar Sahoo et al. *Simple and Effective Masked Diffusion Language Models*. 2024. arXiv: 2406.07524 [cs.CL]. URL: https://arxiv.org/abs/2406.07524.

[16] Yang Song et al. *Score-Based Generative Modeling through Stochastic Differential Equations*. 2021. arXiv: 2011.13456 [cs.LG]. URL: https://arxiv.org/abs/2011.13456.

[17] Jiacheng Ye et al. *Dream 7B*. 2025. URL: https://hkunlp.github.io/blog/2025/dream.

[18] Tianyi Zhang et al. *BERTScore: Evaluating Text Generation with BERT*. 2020. arXiv: 1904.09675 [cs.CL]. URL: https://arxiv.org/abs/1904.09675.

# A Additional Methodology

## A.1 Beta Distribution Visualization

In Figure 3, we visualize the PDFs of tokens at different priority levels.

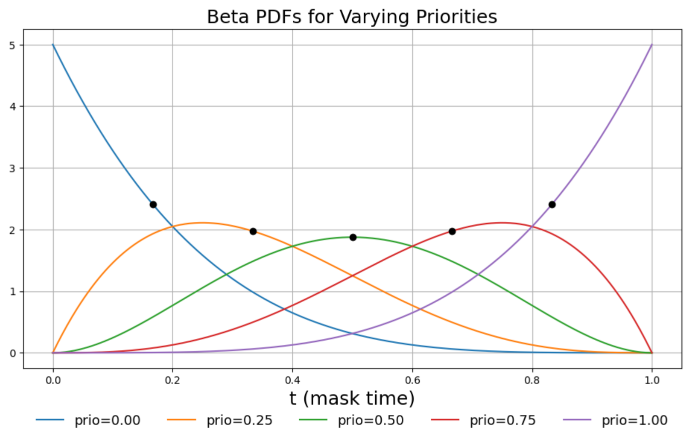


Figure 3: Probability Density Functions (PDFs) of the Beta distribution used for sampling mask times $u_i$ for tokens with varying priorities $p(x_i)$, using Equation 2 with $\gamma = 4$. Higher priority shifts the distribution towards $t = 1$, meaning these tokens are masked later.

## A.2 Assigning Priority Scores

We use Inverse Document Frequency (IDF) as a proxy to assign a priority score $p(x_i)$ to each token in the vocabulary, based on the intuition that rarer words often carry more specific meaning. We calculate IDF scores using a Wikipedia IDF dataset [3] that contains 98.84% of LLaDA English vocabulary tokens.

$$\mathrm{IDF}(x_i) = \log \frac{N}{1 + n(x_i)} \tag{3}$$

where $N$ is the total number of documents in the corpus and $n(x_i)$ is the number of documents containing token $x_i$. These IDF scores are then min-max normalized to the range $[0, 1]$ to serve as the priority scores $p(x_i)$. Figure 4 shows the distribution of these scores over the LLaDA vocabulary.

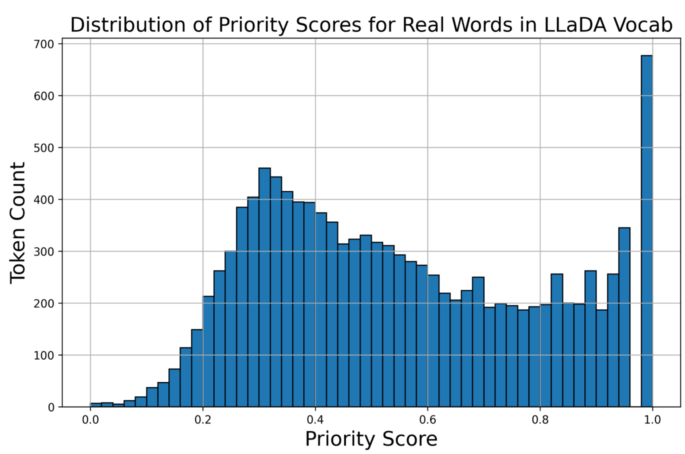


Figure 4: Distribution of normalized IDF priority scores for tokens in the LLaDA vocabulary based on the Wikipedia corpus.

# B Fine-tuning Details

## B.1 Computing Infrastructure

We fine-tuned and performed inference with a single NVIDIA A100 GPU on Yale's Grace HPC cluster. Jobs were submitted via SLURM, requesting one A100 GPU, 4 CPU cores, and 32 GB of RAM for up to 4 hours per run (`-gres=gpu:a100:1, -cpus-per-task=4, -mem=32G, -time=04:00:00`). Fine-tuning was accelerated with bfloat16 mixed precision and TF32 matrix math.

## B.2 Dataset Description

We used a 60k example subset the smoltalk SFT dataset [1].

We first filtered for samples that contained at least one user-assistant response. Then, we took the first such pair among each sample. After this initial filtering, we used a random subset of 60k samples as our training set to fine-tune our model.

## B.3 Hyperparameters and Tuning Process

The model was fine-tuned using a batch size of 1, learning rate of $2e-4$, and trained on 1 epoch of 60k samples with 4 gradient accumulation steps (15k total training steps) and no warmup or decay.

Our max sequence length was 64 using the AdamW optimizer, and we fine-tuned using LoRA with $\alpha = r = 64$ and a dropout of 0.05.

# C Evaluation Details

## C.1 Training Loss Curves

In Figure 5, we present our training loss curves for both the scheduled masking (our model) and baseline.

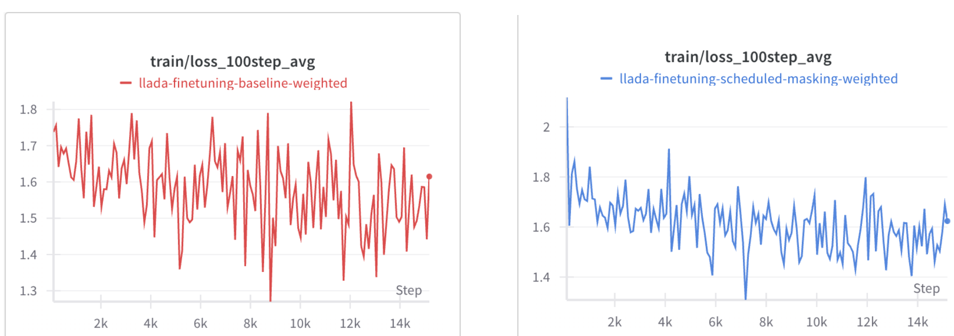


Figure 5: Training losses of both the baseline (left) and PGM (right) during fine-tuning. There is no significant difference other than outlier fluctuations at the beginning of our training. Our final loss is similar, at around 1.6.

## C.2 Remasking Strategies

The two remasking strategies we use in evaluation are random and low-confidence remasking. When we remask randomly, we select 75% of tokens randomly to resample. When we remask based on low confidence, we choose the 75% of tokens with the lowest confidence of tokens selected to remask.

# D Code and Reproducibility

Our code can be found at `https://github.com/ayfeng23/llada_custom_scheduler`, where we include instructions for setting up the experimental environment as well as an `environment.yml` to list all dependencies and external libraries used. Please contact the authors with any issues.